

EPDK Akaryakıt Fiyat Entegrasyonu ve Veri Hattı Tasarımı Teknik ve Hukuki Fizibilite Raporu

Yasal Çerçeve ve Risk Analizi

DepoMetrik projesinin Faz 3 (Çevrimdışı Senkronizasyon, EPDK ve Açık Bankacılık) geliştirme sürecinde, kullanıcıların kredi kartı ekstreleri (PDF) ve banka SMS'lerindeki harcama tutarları üzerinden tüketilen yakıt miktarını (litre) geriye dönük hesaplayabilmesi için Türkiye genelindeki günlük akaryakıt fiyat arşivine ihtiyaç duyulmaktadır. Bu verilerin otomatik sistemlerle (web scraping) toplanması, depolanması ve mobil cihazlarda işlenmesi, Türkiye Cumhuriyeti mevzuatı kapsamında bilişim hukuku, fikri mülkiyet hukuku ve kişisel verilerin korunması mevzuatı çerçevesinde ciddi yasal riskler barındırmaktadır.

Türk Ceza Kanunu Kapsamında Riskler

5237 sayılı Türk Ceza Kanunu'nun (TCK) 243. maddesi "Bilişim sistemine yetkisiz girme ve orada kalma" suçunu, 244. maddesi ise "Sistemi engelleme, bozma, verileri yok etme veya değiştirme" suçunu düzenlemektedir. Dağıtım şirketlerinin (Shell, Opet, Petrol Ofisi, BP) web sitelerinde veya mobil uygulamalarında uyguladıkları API güvenlik duvarlarının, bot engelleme mekanizmalarının (Cloudflare, reCAPTCHA vb.) aşılması veya kullanıcı oturumu gerektiren kapalı alanlara yetkisiz erişim sağlanması TCK Madde 243 kapsamında değerlendirilebilir. Ayrıca, veri çekme botlarının hedef sunuculara çok yüksek frekansta kontrolsüz HTTP istekleri göndermesi, hedef sunucuların bant genişliğini tüketerek sistemlerin yavaşlamasına veya durmasına (DDoS etkisi) sebebiyet verirse, TCK Madde 244 uyarınca bilişim sisteminin işleyişini engelleme suçu oluşabilmektedir.

Fikir ve Sanat Eserleri Kanunu ve Sui Generis Veri Tabanı Hakları

5846 sayılı Fikir ve Sanat Eserleri Kanunu (FSEK) uyarınca, bir internet sitesinde yer alan ve yapılandırılmış verilerden oluşan veri kümeleri "veri tabanı" (database) olarak korunmaktadır. Yargıtay 11. Hukuk Dairesi'nin yerleşik içtihatlarına göre (örneğin Yargıtay 11. HD, E. 2016/6829 K. 2018/768, T. 5.2.2018), bir internet sitesinin veya veri tabanının içeriğinin oluşturulması, doğrulanması ve sunulması için ciddi bir mali, teknik veya beşeri yatırım yapılmış olması durumunda, söz konusu veri tabanı FSEK kapsamında koruma altındadır. Veri tabanının tamamının veya niteliksel/niceliksel açıdan önemli bir kısmının izinsiz olarak sistematik şekilde çekilmesi (extraction) ve başka bir ticari uygulamada kullanılması (re-utilization), "Sui Generis Veri Tabanı Hakları"nın ihlali anlamına gelir ve haksız rekabet teşkil eder.

robots.txt ve Kullanım Koşullarının Hukuki Niteliği

Web sitelerinin kök dizininde bulunan robots.txt dosyaları ve sitelerin "Kullanım Koşulları" (Terms of Service) metinleri, sitelere erişim sağlayan otomatik yazılımlar için yasal sınırları

belirleyen elektronik sözleşmelerdir. robots.txt dosyasındaki kısıtlamaları veya kullanım şartlarındaki "otomatik veri çekme yasağını" (no-scraping clause) ihlal ederek veri kazımak, Türk Borçlar Kanunu (TBK) kapsamında haksız fiil ve sözleşmeye aykırılık teşkil ederek tazminat sorumluluğu doğurabilmektedir.

KVKK ve Veri Minimizasyonu İlkeleri

6698 sayılı Kişisel Verilerin Korunması Kanunu'nun (KVKK) 4. maddesinde yer alan "Amaçla sınırlılık ve veri minimizasyonu" ilkesi gereğince, kullanıcının cihaz üzerindeki bankacılık verileri (SMS ve PDF ekstreler) işlenirken akaryakıt dışı harcamalar, limit bilgileri ve diğer hassas kişisel finansal veriler asla bulut sunucularına gönderilemez. Bu verilerin işlenmesi tamamen cihaz üzerindeki yerel RAM'de (on-device) gerçekleştirilmeli, akaryakıt dışı tüm satırlar anında bellekten silinmeli ve sadece ayıklanan yakıt işlem satırları yerel card_transactions tablosuna yazılmalıdır. Ayrıca, onboarding aşamasında KVKK Madde 5/1 uyarınca katmanlı açık rıza onay mekanizması kurulmalıdır.

Resmi EPDK Veri Altyapısı ve Dağıtım Şirketleri Analizi

Enerji Piyasası Düzenleme Kurumu (EPDK), Türkiye'deki resmi akaryakıt bayi tavan ve satış fiyatlarını günlük olarak yayınlamaktadır. Ancak kurumun web portalı üzerinde doğrudan programatik entegrasyona uygun, yetkilendirme modeli içeren resmi bir JSON/XML REST API endpoint'i bulunmamaktadır. Veriler web portalı üzerinden PDF ve Excel formatlarında manuel olarak indirilebilecek şekilde sunulmaktadır. Türkiye Cumhuriyeti devlet kurumlarının açık veri portalları (örneğin Veri.gov.tr veya belediyelerin açık veri portalları) incelendiğinde, ilçe düzeyinde tarihsel derinliğe sahip ve gerçek zamanlı güncellenen akaryakıt fiyat verisi sağlayan stabil ve resmi bir kamu API'sine rastlanmamaktadır.

Dağıtım Şirketlerinin Arka Plan Altyapılarının Analizi

Dağıtım şirketlerinin (Shell, Opet, Petrol Ofisi, BP) web sitelerindeki fiyat sorgulama sayfaları çoğunlukla istemci taraflı (client-side) dinamik JavaScript kodları ile render edilmektedir. Bu durum, statik HTML parser (örneğin BeautifulSoup veya Cheerio) kullanımını imkansız kılmaktadır. Bu sitelerin ağ (network) trafikleri incelendiğinde, bazı şirketlerin fiyatları çekmek için halka açık ancak dökümantasyonu bulunmayan arka plan API'leri kullandığı görülmüştür:

- **BP API:** https://www.bp.com/bp-tr-pump-prices/api/PumpPrices?strCity={CITY_NAME} URL yapısı üzerinden JSON formatında doğrudan il bazlı veri döndürmektedir.
- **Petrol Ofisi:** JavaScript ile korunan ve dinamik HTML tablolar kullanan bir yapıya sahiptir, doğrudan API erişimi engellenmiş durumdadır.
- **Alpet:** Statik bir HTML yapısı üzerinden fiyatları sunmakta olup, CSS seçicileri ile parse edilmeye elverişlidir.
- **Shell:** Geliştirici portalında (developer.shell.com) B2B müşterileri için "Shell Card Management API" (OAuth 2.0 / API Key korumalı) sunmaktadır ancak bu API sadece kart işlemleri ve kart yönetim verilerine yönelik olup, genel pompa satış fiyatlarını içermemektedir.

Dinamik İçerik Aşma ve Kararlı Web Scraping Stratejileri

Dinamik web sayfalarından veri çekmek için Playwright veya Puppeteer gibi headless tarayıcı motorları tercih edilmelidir. Dağıtım şirketlerinin HTML şemalarındaki tasarım değişikliklerine karşı dirençli (resilient) ve kendi kendini onaran (self-healing) bir kazıma yapısı kurmak amacıyla şu teknik önlemler alınmalıdır:

- **Hiyerarşik Seçici Yedekleme:** Tek bir CSS seçicisine bağımlı kalmak yerine, yedekli seçici listeleri (fallback selectors) tanımlanmalıdır.
- **Metin Bazlı Desen Eşleştirme (Regex):** Fiyat tablosunun yapısı değişse dahi, satır içerisindeki para birimi simgeleri (₺, TL) ve sayısal formatları düzenli ifadelerle yakalayan esnek parser kodları yazılmalıdır.
- **XHR/Fetch İsteklerini Yakalama:** Headless tarayıcıda sayfa yüklenirken, tarayıcının ağ trafiği dinlenmeli ve doğrudan fiyatları içeren ham JSON yanıtları intercept edilmelidir.

Ticari API Sağlayıcıları ve Entegrasyon Çözümleri

Otomatik veri kazıma (scraping) yöntemlerinin operasyonel bakım maliyetleri ve yasal riskleri göz önünde bulundurulduğunda, ticari API sağlayıcılarının birincil veya yedek veri kaynağı olarak kullanılması kaçınılmazdır. Türkiye pazarında bu veriyi sağlayan çözümlerin karşılaştırmalı analizi aşağıdaki tabloda detaylandırılmıştır:

API Sağlayıcı	Kapsam ve Güncelleme Sıklığı	Veri Doğruluk Oranı	Limitler ve Yetkilendirme	Fiyatlandırma Modelleri
ucuzyakitbul.com.tr	81 il ve ilçe bazlı günlük güncellemeler. Doğrudan EPDK verilerini kaynak alır.	%99.8 (Resmi EPDK tavan ve bayi satış fiyatları).	Authorization: Bearer <API_KEY>.	Starter: 500 istek/gün (Ücretsiz) Pro: 10.000 istek/ay (499 ₺/ay) Business: 100.000 istek/ay (1.999 ₺/ay).
CollectAPI (Gas Prices API)	Türkiye geneli il bazlı yakıt fiyatları, elektrikli araç şarj fiyatları.	%95 (Bayi ortalamalarından beslenmektedir).	authorization: apikey <TOKEN>.	Free Trial: 10 istek/ay limitli. Basic: 100 istek/ay (\$14.90/ay) Europe (TR Dahil): Sınırsız istek (\$29.00/ay).
Hasan Adıgüzel API	81 il ve ilçeleri kapsayan günlük güncel veriler.	%92 (Açık kaynaklı gönüllü veri tabanı).	Yetkilendirme bulunmamaktadır (Açık API).	Tamamen Ücretsiz.

Hibrit ve Yedekli (Fallback) Sistem Mimarisi Tasarımı

Herhangi bir ticari API servisinin kesintiye uğraması, DepoMetrik kullanıcılarının yakıt litresi hesaplayamamasına neden olur. Bunu önlemek için, bulut backend tarafında çalışan ve servisleri sırayla deneyen akıllı bir entegrasyon katmanı kurgulanmalıdır. Entegrasyon katmanı

birincil olarak en yüksek doğruluğa sahip ucuzyakitbul.com.tr API'sini sorgular. Eğer bu servisten HTTP 429 (Rate Limit), 500 (Server Error) veya zaman aşımı (Timeout) hatası alınırsa, sistem otomatik olarak CollectAPI servisine istek yönlendirir. Bu servisin de başarısız olması durumunda ücretsiz bir çözüm olan Hasan Adıgüzel API devreye sokulur. En son çare olarak, Deno Edge Function üzerinde barındırılan ve dağıtım şirketlerinin sitelerini doğrudan kazıyan yerel Puppeteer Scraper çalıştırılarak veri hattı tamamlanır.

Çevrimdışı Öncelikli (Offline-First) Veritabanı ve Boyut Optimizasyonu

Türkiye'deki 81 il ve bunlara bağlı yaklaşık 970 ilçede, son 3 yıl boyunca her gün gerçekleşen Benzin, Motorin ve LPG fiyat değişimlerinin saklanması muazzam bir veri yoğunluğu oluşturmaktadır.

Veri Yoğunluğu ve Depolama Sınırları

Ham verilerin mobil cihaz üzerinde ilişkisel bir yapıda saklanması durumunda oluşacak tablonun boyut analizi şu şekildedir:

$\text{Günlük Satır} = 970 \text{ İlçe} \times 3 \text{ Yakıt Türü} = 2.910 \text{ kayıt/gün}$

$\text{Yıllık Toplam Satır} = 2.910 \text{ satır/gün} \times 1.095 \text{ gün} = 3.186.450 \text{ satır}$

Bu hacimdeki bir veri kümesi, SQLite veritabanında indekslerle birlikte yaklaştıkça mobil cihazda yaklaşık 150 MB ila 200 MB arasında bir depolama alanı tüketecektir. Bu durum, DepoMetrik'in hafif ve hızlı çalışma prensibini doğrudan ihlal eder.

Matematiksel Sıkıştırma ve Delta Kodlama Yöntemleri

Veri boyutunu minimize etmek amacıyla uygulanacak optimizasyon yöntemleri:

- **Aralık Kodlaması (SCD Type 2):** Akaryakıt fiyatları her gün değişmemektedir. Fiyatları her gün için tekil birer satır olarak kaydetmek yerine, fiyatın değiştiği tarih aralıklarını saklamak satır sayısını dramatik şekilde düşürür. Bir yılda ortalama 50 fiyat değişimi olduğu varsayıldığında, satır sayısı 3.186.450'den 145.500'e düşerek %95.4 oranında azalır.
- **İlçe-il Delta Kodlaması (District-to-City Delta Encoding):** Türkiye'deki ilçelerin akaryakıt fiyatları, bağlı bulundukları il merkezindeki fiyatlarla doğrudan ilişkilidir ve aralarında genellikle nakliye maliyetlerinden kaynaklanan çok küçük, sabit bir kuruş farkı (Δ_{district}) bulunur. Bu nedenle, zamana bağlı ana fiyat geçmişi yalnızca 81 il bazında saklanır. İlçeler için ise zamandan bağımsız ve sadece il fiyatına eklenecek olan delta farkını tutan statik bir tablo kurgulanır:

$$P_{\text{district}}(t) = P_{\text{province}}(t) + \Delta_{\text{district}}$$

Eğer ilçedeki fiyat il merkeziyle aynıysa, delta değeri 0 olarak kaydedilir. Bu yöntemle temporal veri tablosunun boyutu 1.7 MB seviyesine kadar düşürülebilir.

- **Tam Sayı Dönüşümü (Integer representation):** Kuruş hassasiyetini kaybetmemek ve veritabanı seviyesinde IEEE-754 kayan noktalı sayı (float) hatalarını önlemek için fiyatlar veritabanında tamsayı (INTEGER) cinsinden saklanmalıdır (örneğin 42.15 TL fiyatı veritabanında 4215 kuruş olarak tutulur). Bu yöntem, tamsayı sıkıştırma algoritmalarının (Simple-8b, Run-Length Encoding) veritabanı motoru seviyesinde çok daha yüksek performansla çalışmasını sağlar.

Sunucu ve İstemci Arasındaki Veri Akış Hattı (Pipeline) Tasarımı

Sistem mimarisinde, sunucu tarafında Supabase PostgreSQL veritabanı ve Deno tabanlı Edge Function yapıları konumlandırılmıştır. PostgreSQL tablosu üzerinde taranan günlük güncellemeler, PowerSync gerçek zamanlı senkronizasyon motoru aracılığıyla yerel Drift (SQLite) veritabanına iletilir.

Supabase Edge Function (Deno) Fiyat Çekme ve SCD Type 2 Entegrasyonu

Bu Deno servisi, günlük tetikleyicilerle çalışarak dış kaynaklardan fiyatları çeker ve veritabanında Slowly Changing Dimension (SCD Type 2) mantığıyla saklar.

```
import { serve } from "https://deno.land/std@0.168.0/http/server.ts"
import { createClient } from "https://esm.sh/@supabase/supabase-js@2.21.0"

const SUPABASE_URL = Deno.env.get("SUPABASE_URL") ?? "";
const SUPABASE_SERVICE_ROLE_KEY =
  Deno.env.get("SUPABASE_SERVICE_ROLE_KEY") ?? "";
const API_KEY = Deno.env.get("UCUZ_YAKIT_BUL_API_KEY") ?? "";

const supabase = createClient(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY);

serve(async (req) => {
  try {
    // ucuzyakitbul API'den tüm istasyonların ve fiyatların güncel halini çek
    const response = await fetch("https://ucuzyakitbul.com.tr/api/stations?limit=500", {
      headers: { "Authorization": `Bearer ${API_KEY}` }
    });

    if (!response.ok) {
      throw new Error(`Dış API bağlantı hatası: ${response.status}`);
    }

    const data = await response.json();
    const stations = data.stations;
    const today = new Date().toISOString().split("T");

    for (const station of stations) {
      const city = station.city.toUpperCase();
      const district = station.district.toUpperCase();

      for (const fuel of station.fuelPrices) {
        const fuelType = fuel.fuelType.toUpperCase(); // BENZİN, MOTORİN, LPG
        const priceInKurus = Math.round(fuel.price * 100);
```

```

// Mevcut aktif kaydı sorgula
const { data: activeRecord, error: selectError } = await supabase
  .from("daily_prices")
  .select("id, price_kurus")
  .eq("city", city)
  .eq("district", district)
  .eq("fuel_type", fuelType)
  .is("valid_to", null)
  .maybeSingle();

if (selectError) continue;

if (activeRecord) {
  // Fiyat değişmediyse pas geç
  if (activeRecord.price_kurus === priceInKurus) {
    continue;
  }

  // Fiyat değiştiyse, eski kaydı kapat (SCD Type 2)
  await supabase
    .from("daily_prices")
    .update({ valid_to: today })
    .eq("id", activeRecord.id);
}

// Yeni fiyat kaydı aç
await supabase
  .from("daily_prices")
  .insert({
    city: city,
    district: district,
    fuel_type: fuelType,
    price_kurus: priceInKurus,
    valid_from: today,
    valid_to: null
  });
}

return new Response(JSON.stringify({ success: true }), { status: 200 });
} catch (error: any) {
  return new Response(JSON.stringify({ error: error.message }), { status: 500 });
}
});

```

Dart ve Drift (SQLite) Yerel Veritabanı Mimarisi

DepoMetrik mobil uygulamasında, Clean Architecture standartlarına tam uyumlu, tip güvenli Drift ORM ve SQLCipher şifreleme altyapısına sahip yerel veritabanı katmanı şu şekilde tasarlanmıştır:

```
import 'package:drift/drift.dart';
```

```
part 'local_database.g.dart';
```

```
/// DepoMetrik Yakıt Tipleri Enum Yapısı
```

```
enum FuelType { benzin, dizel, lpg, elektrik }
```

```
/// profiles tablosu
```

```
class Profiles extends Table {
```

```
  TextColumn get userId => text(); // UUID - PK
```

```
  TextColumn get email => text().nullable();
```

```
  DateTimeColumn get createdAt => dateTime();
```

```
  BoolColumn get premiumStatus => column().withDefault(const Constant(false));
```

```
  @override
```

```
  Set<Column> get primaryKey => {userId};
```

```
}
```

```
/// vehicles tablosu
```

```
class Vehicles extends Table {
```

```
  TextColumn get vehicleId => text(); // UUID - PK
```

```
  TextColumn get userId => text().references(Profiles, #userId());
```

```
  TextColumn get plate => text().withLength(min: 5, max: 15());
```

```
  TextColumn get brand => text();
```

```
  TextColumn get model => text();
```

```
  IntColumn get fuelType => intEnum<FuelType>();
```

```
  IntColumn get initialOdometer => integer().withDefault(const Constant(0));
```

```
  IntColumn get currentOdometer => integer().withDefault(const Constant(0));
```

```
  @override
```

```
  Set<Column> get primaryKey => {vehicleId};
```

```
}
```

```
/// stations tablosu
```

```
class Stations extends Table {
```

```
  TextColumn get stationId => text(); // UUID - PK
```

```
  TextColumn get brandName => text();
```

```
  RealColumn get latitude => real();
```

```
  RealColumn get longitude => real();
```

```
  TextColumn get city => text();
```

```
  TextColumn get district => text();
```

```
@override
Set<Column> get primaryKey => {stationId};
}
```

/// refuelings tablosu

```
class Refuelings extends Table {
  TextColumn get refuelingId => text(); // UUID - PK
  TextColumn get vehicleId => text().references(Vehicles, #vehicleId, onDelete:
  KeyAction.cascade());
  TextColumn get stationId => text().nullable().references(Stations, #stationId, onDelete:
  KeyAction.setNull());
  RealColumn get liters => real();
  RealColumn get unitPrice => real();
  RealColumn get totalPrice => real();
  IntColumn get odometer => integer().withDefault(const Constant(0));
  DateTimeColumn get purchaseDate => dateTime();
  BoolColumn get isFullTank => column().withDefault(const Constant(true));
}
```

```
@override
Set<Column> get primaryKey => {refuelingId};
}
```

/// card_transactions tablosu

```
class CardTransactions extends Table {
  TextColumn get transactionId => text(); // UUID - PK
  TextColumn get userId => text().references(Profiles, #userId);
  TextColumn get refuelingId => text().nullable().references(Refuelings, #refuelingId, onDelete:
  KeyAction.setNull());
  DateTimeColumn get transactionDate => dateTime();
  RealColumn get amount => real();
  TextColumn get merchantName => text();
  TextColumn get source => text(); // PDF, SMS, API
}
```

```
@override
Set<Column> get primaryKey => {transactionId};
}
```

/// campaigns tablosu

```
class Campaigns extends Table {
  TextColumn get campaignId => text(); // UUID - PK
  TextColumn get userId => text().references(Profiles, #userId);
  TextColumn get bankName => text();
  TextColumn get stationBrand => text();
  IntColumn get targetTxCount => integer();
  IntColumn get currentTxCount => integer().withDefault(const Constant(0));
  RealColumn get rewardAmount => real();
  DateTimeColumn get expiryDate => dateTime();
}
```

```

    @override
    Set<Column> get primaryKey => {campaignId};
}

/// obd_readings tablosu
class ObdReadings extends Table {
    TextColumn get readingId => text(); // UUID - PK
    TextColumn get vehicleId => text().references(Vehicles, #vehicleId, onDelete:
    KeyAction.cascade());
    IntColumn get odometerValue => integer();
    RealColumn get fuelLevelRatio => real();
    DateTimeColumn get recordedAt => dateTime();

    @override
    Set<Column> get primaryKey => {readingId};
}

/// Cihaz içi sıkıştırılmış yakıt fiyatlarını tutan optimize tablo
class LocalFuelPrices extends Table {
    IntColumn get id => integer().autoIncrement();
    TextColumn get city => text();
    TextColumn get district => text();
    IntColumn get fuelType => intEnum<FuelType>();
    IntColumn get priceInKurus => integer();
    DateTimeColumn get validFrom => dateTime();
    DateTimeColumn get validTo => dateTime().nullable();
}

@DriftDatabase(tables:)
class AppDatabase extends _$AppDatabase {
    AppDatabase(QueryExecutor e) : super(e);

    @override
    int get schemaVersion => 1;
}

```

Tüketim Hesaplama Motoru (ConsumptionCalculator) ve Matematiksel Doğrulama

Hesaplama motoru, cihaz yerel veritabanında depolanan fiyat ve sayaç verilerini kullanarak "Depodan Depoya" ve "Kayan Ağırlıklı Ortalama" tüketim algoritmalarını işletir.

- **Depodan Depoya (Full-to-Full) Formülü:** İki ardışık tam depo alımı ($is_full_tank = true$) arasındaki tüketimi hesaplar :

$$C_{i[span_64]}(start_span)[span_64](end_span) = \frac{L \times 100}{K_i - K_{i-1}}$$
- **Kayan Ağırlıklı Ortalama (Rolling Average) Formülü:** Kısmi alımların yer aldığı izleme periyotlarındaki tüketimi hesaplar :

$$C_{\text{rolling}} = \frac{\left(\sum_{j=1}^n L_j \right) \times 100}{K_n - K_0}$$

Burada kilometrenin kronolojik olarak sürekli artan (strictly increasing, $K_i > K_{i-1}$) yapıda olması zorunludur. Aksi halde InvalidOdometerException fırlatılmalıdır.

```
import 'dart:math';
```

```
/// Kilometre sayaç hatası durumunda fırlatılacak özel istisna sınıfı
```

```
class InvalidOdometerException implements Exception {
  final String message;
  InvalidOdometerException(this.message);
  @override
  String toString() => 'InvalidOdometerException: $message';
}
```

```
class ConsumptionCalculator {
```

```
  /// Depodan Depoya (Full-to-Full) Tüketim Hesaplama Fonksiyonu
```

```
  static double calculateFullToFull({
```

```
    required double liters,
```

```
    required int currentOdometer,
```

```
    required int previousOdometer,
```

```
  }) {
```

```
    if (currentOdometer <= previousOdometer) {
```

```
      throw InvalidOdometerException(
```

```
        "Mevcut kilometre değeri ($currentOdometer), önceki kilometre değerinden  
($previousOdometer) küçük veya eşit olamaz."
```

```
      );
```

```
    }
```

```
    // Formül:  $C_i = (\text{Litre} * 100) / (K_i - K_{i-1})$ 
```

```
    return (liters * 100) / (currentOdometer - previousOdometer);
```

```
  }
```

```
  /// Kayan Ağırlıklı Ortalama (Rolling Average) Tüketim Hesaplama Fonksiyonu
```

```
  static double calculateRollingAverage({
```

```
    required List<double> litersList, // İlk yakıt alımı (K_0) hariç, sonraki tüm alımların litreleri
```

```
    required int currentOdometer, // K_n değeri
```

```
    required int initialOdometer, // K_0 değeri
```

```
  }) {
```

```
    if (currentOdometer <= initialOdometer) {
```

```
      throw InvalidOdometerException(
```

```
        "Mevcut kilometre değeri ($currentOdometer), başlangıç kilometre değerinden  
($initialOdometer) küçük veya eşit olamaz."
```

```
      );
```

```
    }
```

```
    if (litersList.isEmpty) {
```

```
      return 0.0;
```

```
    }
```

```
final double totalLiters = litersList.reduce((value, element) => value + element);

// Formül: C_rolling = (Toplam Litre (K0 hariç) * 100) / (K_n - K_0)
return (totalLiters * 100) / (currentOdometer - initialOdometer);
}
}
```

Sonuç ve Faz 3 Yol Haritası Eylemleri

DepoMetrik projesinin 9-12. haftalarını kapsayan **Faz 3 (Çevrimdışı Senkronizasyon, EPDK ve Açık Bankacılık)** hedefleri doğrultusunda, geliştirme ekibinin uygulaması gereken somut mimari adımlar aşağıda maddeler halinde sunulmuştur:

1. PowerSync Entegrasyonu ve Veri Eşitleme Yapılandırması

Supabase PostgreSQL üzerinde çalışan daily_prices tablosundaki verilerin, PowerSync senkronizasyon kuralları (sync rules) aracılığıyla mobil istemcilere aktarılması sağlanmalıdır. PowerSync, sadece değişen verileri (CDC logları üzerinden) mobil cihaza sıkıştırılmış JSON diff'leri halinde gönderecek şekilde konfigüre edilmelidir. Bu sayede, hücresel veri tüketimi minimum seviyede tutulacaktır.

2. Deno Edge Function ve pg_cron Kurulumu

Supabase veritabanında her gece 00:05'te çalışacak bir pg_cron tetikleyicisi kurulmalı ve hazırlanan Deno Edge Function çağrılmalıdır. Bu Edge Function, birincil veri kaynağı olan ucuzyakitbul.com.tr API'sini sorgulayacak, arıza durumunda ise CollectAPI ve Hasan Adıgüzel API yedek katmanlarını devreye sokacaktır.

3. Drift Yerel DB Şema Değişikliği (Migration)

Cihaz üzerindeki Drift SQLite yapısına LocalFuelPrices tablosu eklenerek bir veritabanı migrasyonu gerçekleştirilmelidir. Fiyatların INTEGER kuruş formatında depolanması sağlanarak tamsayı veri sıkıştırması optimize edilmeli ve hassas finansal hesaplamalarda yuvarlama hatalarının önüne geçilmelidir.

4. Açık Bankacılık ve QPAY Entegrasyon Hazırlığı

ÖHVPS 2.0 (Ödeme Hizmetleri Veri Paylaşım Servisi) standartlarına uygun olarak, entegratör QPAY servis dökümantasyonuna göre BKM GEÇİT yönlendirme arayüzleri tasarlanmalıdır. Kullanıcı rızasıyla alınacak olan hesap hareketlerindeki akaryakıt işlemlerinin, cihaz üzerinde lokal olarak çözümlenen birim fiyatlarla eşleştirilerek net litre tespiti yapılması süreci test senaryolarıyla doğrulanmalıdır.

Bu teknik, hukuki ve matematiksel altyapı sayesinde DepoMetrik uygulaması, tamamen çevrimdışı çalışabilen, veri boyutu açısından optimize edilmiş ve KVKK ilkeleriyle %100 uyumlu

bir akaryakıt takip ve tüketim yönetim sistemine kavuşacaktır.

Alıntılanan çalışmalar

1. Yapay Zeka Hukuku | AI Act, KVKK, Telif, Sorumluluk 2026 | Av. Bilal ALYAR, <https://bilalalyar.av.tr/yapay-zeka-hukuku-72970/>
2. Web Scraping/Kazıma ve İnternet Sitelerinin Bundan Korunması, <https://www.guleryuz.av.tr/tr/news-publications/detail/https-www-guleryuzavtr-tr-news-publication-s-detail-web-kazima-web-scraping-ve-internet-sitelerine-yonelik-koruma>
3. Web scraping yasal mı? : r/webdev - Reddit, https://www.reddit.com/r/webdev/comments/1ain86a/is_web_scraping_legal/?tl=tr
4. WEB SCRAPING EYLEMİNİN HAKSIZ REKABET AÇISINDAN DEĞERLENDİRİLMESİ, <https://www.goksusafiisik.av.tr/tr/publications/2025-summer-issue/web-scraping-eyleminin-haksiz-rekabet-acisindan-degerlendirilmesi?id=510>
5. EPDK - Akaryakıt Bayi Satış Fiyatları - Günlük - Hakediş.org, <https://www.hakedis.org/endeksler/epdk-akaryakit-bayi-satis-fiyatları>
6. Akaryakıt Fiyatları Nasıl Oluşur? - Opet, <https://www.opet.com.tr/akaryakit-fiyatları-nasil-olusur>
7. Akaryakıt Fiyatları - Petrol Ofisi, <https://www.petrolofisi.com.tr/akaryakit-fiyatları>
8. eklenecekler · Issue #1 · ademilter/akaryakit-fiyatları - GitHub, <https://github.com/ademilter/akaryakit-fiyatları/issues/1>
9. API Catalog | Shell Developer Portal, <https://developer.shell.com/api-catalog>
10. Akaryakıt Fiyatları API - CollectAPI, <https://collectapi.com/tr/api/gasPrice/akaryakit-fiyatları-api>
11. Yakıt Fiyatları API — Türkiye'nin En Kapsamlı Akaryakıt Verisi ..., <https://ucuzyakitbul.com.tr/api-docs>
12. Gas Prices API - CollectAPI, <https://collectapi.com/api/gasPrice/gas-prices-api?tab=pricing>
13. Güncel Akaryakıt Fiyatları API | Hasan Adıgüzel, <https://www.hasanadiguzel.com.tr/api-tutorials/guncel-akaryakit-fiyatları-api>
14. Data Compression Techniques for Time Series - QuestDB, <https://questdb.com/glossary/data-compression-techniques-for-time-series/>
15. Time-series compression algorithms, explained | Tiger Data, <https://www.tigerdata.com/blog/time-series-compression-algorithms-explained>
16. orbitinghail/sqlite-delta: Lightweight CDC patterns for SQLite - GitHub, <https://github.com/orbitinghail/sqlite-delta>